DALHOUSIE UNIVERSITY

Preprocessing and Visualizing Data

Ordered observations, graphical integrity, and amounts

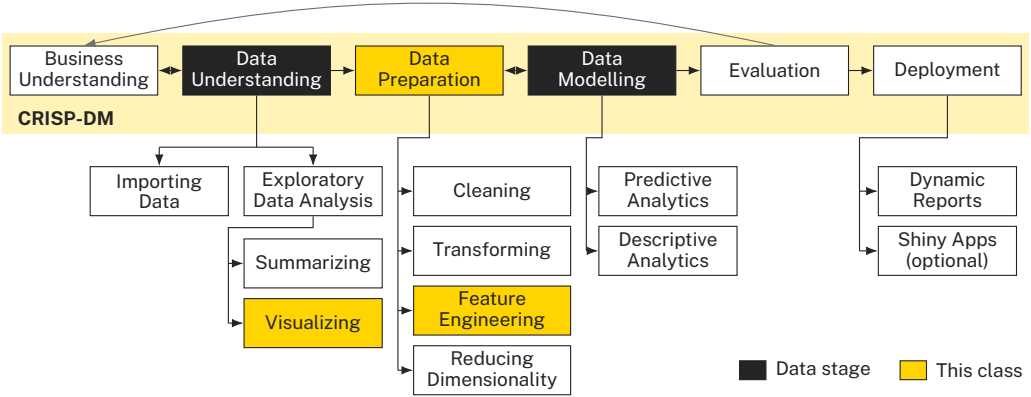
Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Continuing onward

- Bring your Lab 02 cleaning script and its record of decisions. Lab 03 applies them to a new dataset.
- Practical work continues in the scheduled Class B meeting.
- Use R or Python. The notes include complete setup for each example.

Keep assessed work private until the deadline and instructor release.

Features and Visual Exploration



We move between preparation and exploration as the data raises new questions.

Give a New Variable a Meaning

A useful derived column expresses a relationship that matters to the question.

Examples include an amount per person, a previous measurement, or a change from one interval to the next.

Record the Context of a Measurement

- A sensor reading may depend on what was measured a minute earlier.
- Repeated measurements from one person share a subject identity.
- Nearby places may share conditions. A location code alone does not define a neighbourhood.

State what each row represents before deriving a contextual variable.

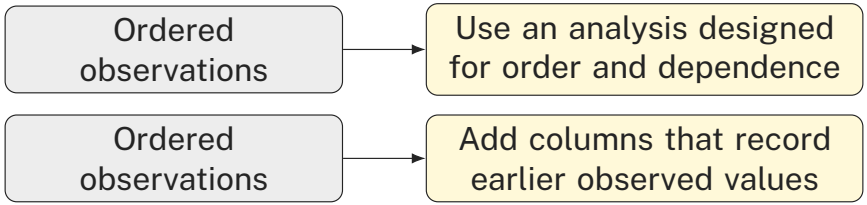
Order and Identity Matter

Columns record measured variables
Rows identify observations

Time order and subject identity
are part of the data's meaning.
Keep both explicit.

A row is an observation, not a promise of independence. Shuffling a time series can hide the very pattern we want to examine.

Two Complementary Approaches



The new columns preserve context. Dependence can remain.

Today we construct context columns. Choosing an appropriate model comes later.

A Four-Column Time Delay Embedding

One subject has seven equally spaced, chronologically ordered readings, $4, 5, 6, 7, 8, 9, 10$.

Time t	Y_t	Y_{t-1}	Y_{t-2}	Y_{t-3}
4	7	6	5	4
5	8	7	6	5
6	9	8	7	6
7	10	9	8	7

Each row contains the current value and its three preceding values. The first three times lack enough history, leaving four complete rows.

Build the Lag Columns

Python starts with the imports in the notes.

R

```r
# One subject, equally spaced times, already
# in chronological order.
y <- c(4, 5, 6, 7, 8, 9, 10)
lags <- embed(y, dimension=4)
colnames(lags) <- c("Y", "lag1", "lag2", "lag3")
print(lags)
# Rows 7,6,5,4; 8,7,6,5; 9,8,7,6; 10,9,8,7.
```

Python

```python
# One subject, equally spaced times, already
# in chronological order.
y = pd.Series([4, 5, 6, 7, 8, 9, 10],
              dtype=float)
lags = pd.DataFrame({
    "Y": y, "lag1": y.shift(1),
    "lag2": y.shift(2), "lag3": y.shift(3),
}).dropna()
print(lags.to_numpy())
# Rows [7,6,5,4], [8,7,6,5], [9,8,7,6],
# [10,9,8,7].
```

`embed` keeps the input type, so the R matrix stores doubles. Sort within each subject first; a row shift is a fixed duration only for equally spaced measurements.

Distinguish an Amount from a Rate of Change

For an amount with a meaningful ratio scale and a nonzero previous value, the fractional relative change is

$$r_t = \frac{Y_t - Y_{t-1}}{Y_{t-1}}.$$

Multiply by 100 to express the fractional change as a percentage.

A zero or missing previous value requires an explicit handling rule.

$$6 \longrightarrow 9$$

$$(9 - 6)/6 = 0.5 = 50\%$$

Constant Proportional Growth

Python continues from the imports in the notes.

R

```r
y <- 4 * 1.5^(0:5)
stopifnot(all(!is.na(y)), all(y > 0))
change <- diff(y) / head(y, -1)
print(change)  # Five values equal to 0.5.
print(100 * change)  # 50 percent each interval.
```

Python

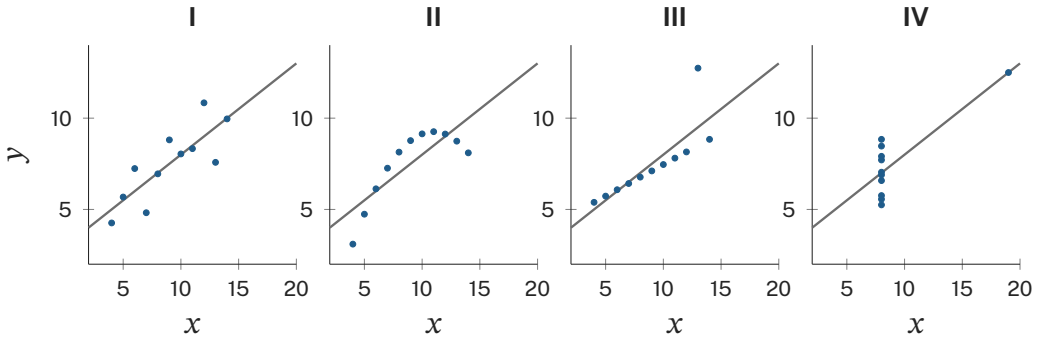
```python
y = pd.Series(4 * 1.5 ** np.arange(6))
assert y.notna().all() and (y > 0).all()
change = y.pct_change(fill_method=None)
print(change.tolist())
# Missing first value, then five values
# equal to 0.5.
print((100 * change).tolist())
# Missing first value, then 50 percent each
# interval.
```

Constant relative change in this sequence does not mean differencing removes trends or dependence from real data. R returns five changes; pandas keeps six time positions, the first missing.

Look at the Values as Well as the Summaries

A mean, a standard deviation, and a correlation describe selected features of a dataset. A plot can reveal what those numbers leave out.

Anscombe's Quartet



Each panel contains the original eleven points. The grey line $y = 3 + 0.5x$ is the rounded common fit, shown for comparison.[1]

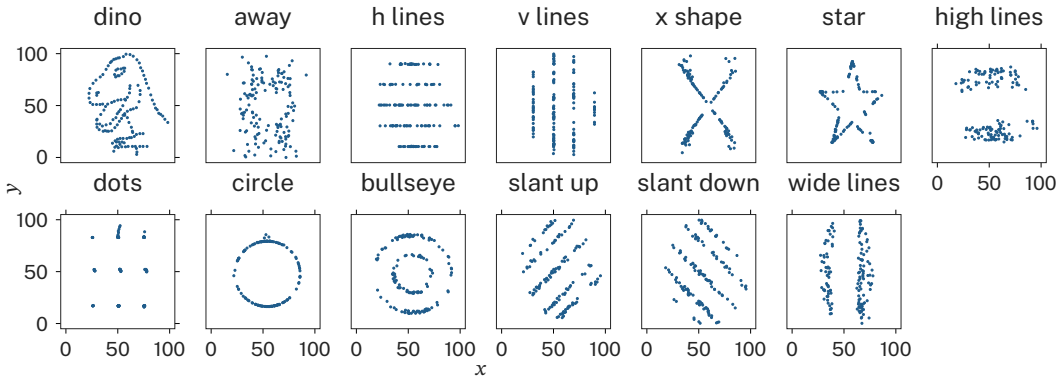
Similar Summaries, Different Patterns

Summary	Rounded value in each set
Mean x; sample SD x	9.00; 3.32
Mean y; sample SD y	7.50; 2.03
Pearson correlation r	0.82
Approximate fitted line	$y = 3 + 0.5x$

- I shows scatter around a straight pattern; II shows curvature.
- III has an unusual point above an otherwise near-linear pattern.
- IV has ten points at $x = 8$ and one at $x = 19$.

The summaries agree at the displayed precision. They are not identical at every precision.

The Datasaurus Dozen



Thirteen authentic datasets, 142 points each, on common axes.

Original dinosaur by Alberto Cairo; other shapes by Matejka and Fitzmaurice [3].

Nearly Equal Numbers, Distinct Shapes

Summary	Range across 13 sets
Mean x	54.26 to 54.27
Mean y	47.83 to 47.84
Sample SD x	16.76 to 16.77
Sample SD y	26.93 to 26.94
Pearson r	-0.07 to -0.06

These ranges are calculated from the original 1 846 coordinates. The small negative correlations describe linear association; they do not rule out a clear nonlinear pattern.

Use Both Calculations and Graphs

- Ask what a summary measures and what it ignores.
- Inspect shape, gaps, subgroups, and unusual observations.
- Return to the data dictionary before treating an unusual point as an error.
- Report a pattern as evidence about these data, with its limits.

The notes reproduce both published examples in R and Python and link directly to the Datasaurus data.

Start from the Question

DALHOUSIE UNIVERSITY

Question	Starting points
How much in each category?	Bars, dots, grouped or stacked bars, heatmaps.
How are values distributed?	Histograms, densities, box plots, violins, ECDFs, Q-Q plots.
How is a whole divided?	Bars, pies, stacked bars; stacked areas over time.
How do two variables relate?	Scatter plots; a line when order has meaning.
How do several variables relate?	Scatter-plot matrices, correlograms, parallel coordinates.

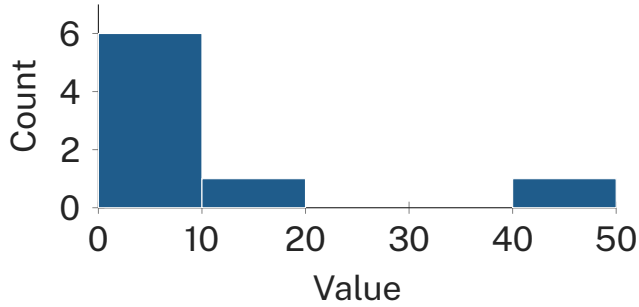
Choose from the question, variable type, units, and meaningful order. The same numeric column may support different questions.

Four Libraries Make Different Initial Choices

Interface	Default bin choice
R `hist`	Sturges' rule, then convenient break locations.
ggplot2 `geom_histogram`	30 bins if no bin choice is supplied.
Matplotlib `hist`	10 bins under the standard configuration.
seaborn `histplot`	NumPy's `auto` bin rule.

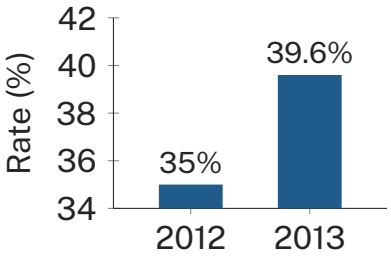
Defaults are starting points. For a deliberate cross-language comparison, use the same observed values, explicit edges, and missing-value policy.

A Reproducible Histogram Comparison



Values $2, 4, 4, 5, 7, 9, 12, 41$ and one missing entry. Explicit edges $0, 10, 20, 30, 40, 50$ give counts $6, 1, 0, 0, 1$ in both languages. The complete code is in the notes.

What Does a Clipped Bar Imply?

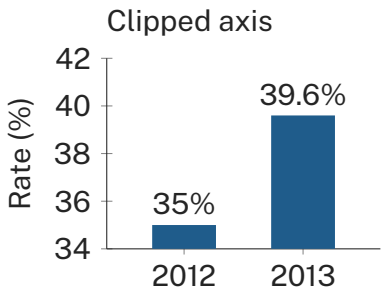


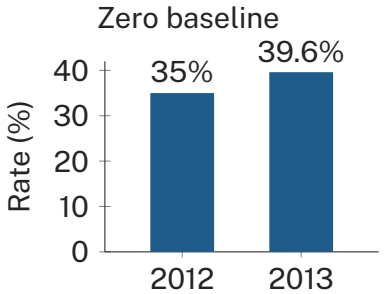
A teaching redraw compares the 2012 top tax rate, 35%, with the 39.6% scheduled for 2013.

The difference is 4.6 percentage points. Relative to 35%, the increase is about 13.1%.

The visible bar lengths start at 34%, making the second appear 5.6 times as long.

Show the Same Values from Zero





Both show an increase. A zero baseline makes bar length proportional to the rate being compared.

Four Questions Before Accepting a Plot

Match visual magnitude to numeric magnitude.	For bars, begin at zero so length represents the stated amount.
Label the comparison completely.	Give the variable, unit, population, time period, and source.
Show variation in data, not decoration.	Keep scales and encodings consistent across comparable panels.
Give enough context to interpret the result.	Report exclusions and avoid selecting a range that conceals the pattern.

A clear chart makes the comparison, and the choices that shaped it, easier to inspect. Adapted as course design principles from Tufte.[4]

Choose an Encoding for the Task

Compare numeric values

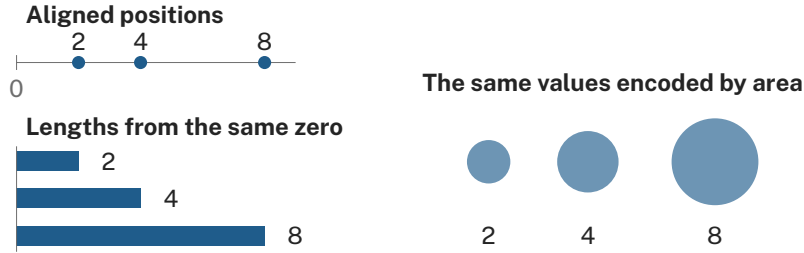
Aligned position and length usually support more precise comparisons than area, angle, or intensity.

Identify groups

Use hue, shape, line style, or text to distinguish groups. Categories do not need a magnitude ranking.

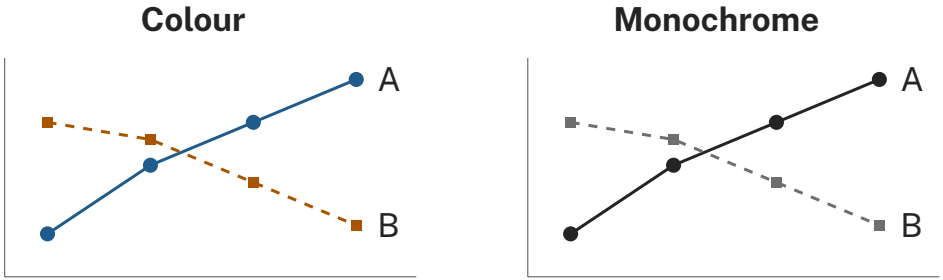
A perceptual comparison of quantities is a different task from identifying which group a mark belongs to.

Compare the Same Three Values



The last circle has four times the area of the first, a ratio harder to judge than aligned lengths. Aligned position or length usually supports more precise comparisons.

Keep Meaning When Colour Changes



A and B remain identifiable through direct labels, circles versus squares, and solid versus dashed lines. This is a monochrome check, not a simulation of a particular visual condition.

What Should the Reader Compare?

Use bars or dots to compare amounts across categories. Use grouped or stacked bars when the question includes both categories and their components.

Start from Data, Marks, and Labels

- Data, five teams' regular-season wins in the Atlantic Division.
- Mapping, team on the category axis and wins on the numeric axis.
- Mark, a bar or a dot for each team.
- Labels and scales, season, unit, readable names, and a deliberate range.

The full twenty-value table and both language setups are in the notes before the plotting code.

Express the Same Design in Either Language

DALHOUSIE UNIVERSITY

R

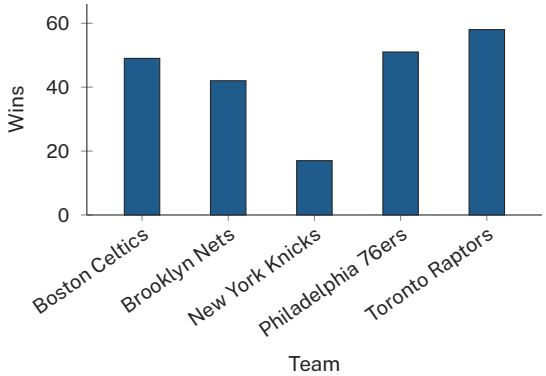
`ggplot` specifies data and mappings. A layer such as `geom_col` supplies the marks. Labels, scales, and a theme complete the plot.

Python

A Matplotlib figure contains axes. `ax.barh` supplies bars on those axes. Axis methods add the labels, limits, and title.

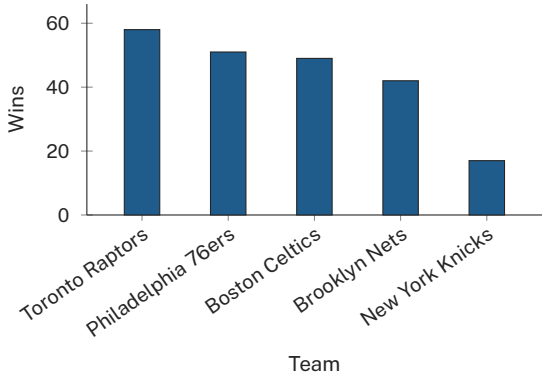
These are two implementations of the same visual choices. Python also has layered interfaces; no language owns the underlying idea.

Alphabetical Order with Rotated Labels



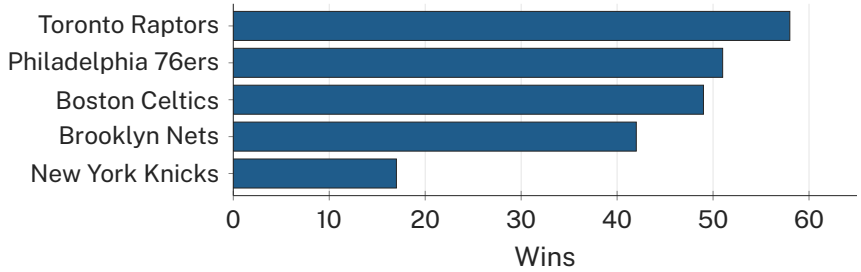
Alphabetical order supports lookup. The long team names fit only when rotated, so each one is read at an angle.

Order by the Quantity You Compare



Ordering by wins supports ranking, but the names are still rotated. Keep inherent order for ordinal categories or time.

Turn the Bars to Make Names Readable



Toronto has 58 wins and New York has 17 in 2018–19. The largest value appears at the top and all names fit.

Build an Ordered Horizontal Bar Chart

Continue from the NBA setup in the notes.

R

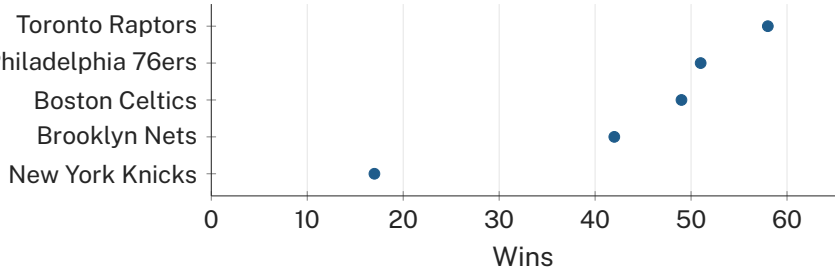
```r
p <- ggplot(ordered, aes(x=`2018-19`, y=Team)) +
  geom_col(fill=blue) + xlim(0, 65) +
  labs(x="Wins", y=NULL,
       title="Atlantic Division, 2018-19") +
  theme_minimal()
ggsave("class-05-bars-r.pdf", p, width=7,
       height=4)
```

Python

```python
fig, ax = plt.subplots()
ax.barh(ordered["Team"], ordered["2018-19"],
        color=blue)
ax.set(xlabel="Wins",
       title="Atlantic Division, 2018-19",
       xlim=(0, 65))
fig.tight_layout()
fig.savefig("class-05-bars-python.pdf")
plt.close(fig)
```

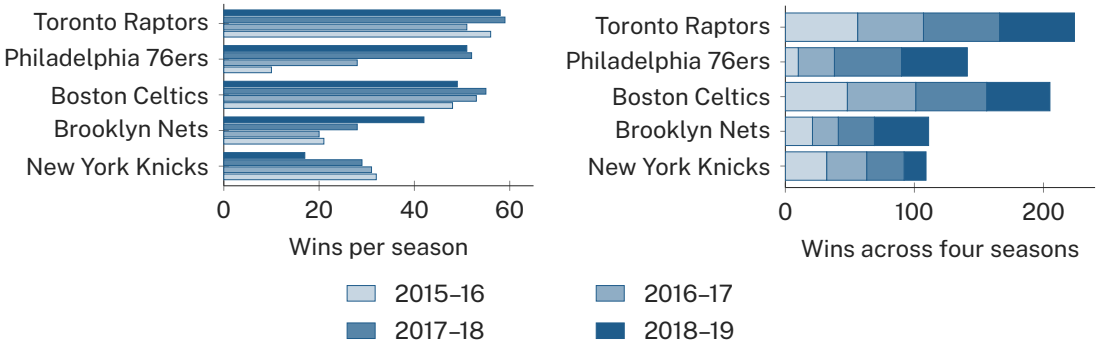
`ordered` stores the teams from smallest to largest, as R factor levels. ggplot2 and Matplotlib both place the first category at the bottom, so the largest team appears at the top in both languages.

Compare Positions on a Common Scale



Dots represent the same five wins. Position carries the comparison, so a carefully labelled narrower range can be useful when values are close. This view keeps the same 0–65 scale as the bars.

Grouped and Stacked Bars Differ



Grouped bars compare seasons from zero; stacks compare totals. Seasons run from 2015–16, lightest, to 2018–19, darkest, bottom to top within groups and left to right within stacks.

Control Grouping and Stacking

Continue from the long-table (R) and wide-table (Python) setups in the notes.

R

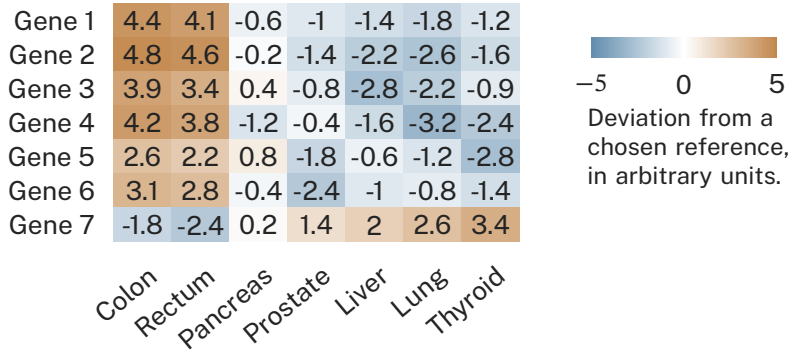
```r
p <- ggplot(nba_long,
            aes(x=Wins, y=Team, fill=Season)) +
  geom_col(
    position=position_stack(reverse=TRUE),
    width=.7) +
  scale_fill_manual(values=colors,
                    name="Season\n(left to right)") +
  xlim(0, 240) +
  labs(x="Wins across four seasons", y=NULL) +
  theme_minimal()
ggsave("class-05-stacked-r.pdf", p, width=8,
       height=5)
```

Python

```python
ax = wide.plot.barh(stacked=True, color=colors,
                    width=0.7)
ax.set(xlabel="Wins across four seasons",
       xlim=(0, 240))
ax.legend(title="Season (left to right)",
          ncols=1, loc="upper left",
          bbox_to_anchor=(1, 1))
ax.figure.tight_layout()
ax.figure.savefig("class-05-stacked-python.pdf")
plt.close(ax.figure)
```

`reverse=TRUE` orders seasons left to right in time; each season is a column of `wide`. Grouped bars use `position_dodge` in R and omit `stacked=True` in Python.

Read a Numeric Matrix Through Colour



Each cell prints its deviation from the reference.

Layout after Choi (2015), Wikimedia Commons [2].

A Colour Scale Does Not Require Clustering

- Each cell maps one numeric value to a colour. White is fixed at the reference value zero in this example.
- Rows and columns retain the stated order. The notes include the complete matrix and both plotting implementations.
- A clustered heatmap can reorder similar rows or columns and add a tree. The result depends on the similarity and grouping rules.

Check Meaning, Evidence, and Presentation

- Derived variables need a clear definition and valid inputs.
- Lag columns preserve history; relative changes need a meaningful baseline.
- Plots and summaries describe different aspects of the same data.
- Make the question, amount, unit, order, and grouping visible.
- Use readable labels and more than colour to identify groups.

Sources Cited in This Class

[1] F. J. Anscombe. “Graphs in statistical analysis”. In: *The American Statistician* 27.1 (1973), pp. 17–21.

[2] E.-S. Choi. *Heat Maps of RNAseqV2*. Wikimedia Commons, file Heatmap_RNAseqV2_1.png. CC BY-SA 4.0. 2015.

[3] J. Matejka and G. Fitzmaurice. “Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. ACM, 2017, pp. 1290–1294.

[4] E. R. Tufte. *The Visual Display of Quantitative Information*. 2nd ed. Graphics Press, 2001.